INF560 / Unidad 3 / GL10

GUÍA DE LABORATORIO

Autorización: roles y permisos

Policies, Gates y el dueño de cada evento en eventos-app

Laravel 13 · Policies · Gates · PostgreSQL 18 · Proyecto acumulativo eventos-app · Duración 3 h

Métodos HTTP de la serie: GET POST PUT DELETE

Universidad Autónoma Tomás Frías · Facultad de Ciencias Puras · Ingeniería Informática

00 Antes de empezar

Tras la GL09, `eventos-app` ya distingue quién ha iniciado sesión, pero **cualquier usuario autenticado puede editar o borrar los eventos de cualquier otro**. Falta la **autorización**: decidir qué puede hacer cada usuario. En esta guía haremos que **solo el dueño** de un evento pueda modificarlo, y que un **administrador** pueda gestionar todos.

> **CIERRE DEL BLOQUE MONOLÍTICO**
> Esta es la última guía del segundo parcial sobre Laravel monolítico. Con ella, `eventos-app` queda como una aplicación completa: datos, CRUD, validación, relaciones, autenticación y **autorización**.

Objetivos de la guía

- Agregar un **rol** a los usuarios (usuario / administrador) mediante una migración.
- Crear usuarios de prueba con un **seeder**, sin usar Tinker.
- Escribir una `EventoPolicy` que autorice según la propiedad del evento.
- Aplicar la autorización en el controlador con `$this->authorize()`.
- Mostrar u ocultar acciones en las vistas con `@can`.
- Definir un **Gate** para un permiso global y proteger un panel de administración.

Requisitos previos

- La GL09 completa: autenticación funcionando, la columna `user_id` en la tabla `eventos` y la relación `usuario()` en el modelo `Evento`.
- El método `store()` de `EventoController` guarda el dueño del evento (`user_id = auth()->id()`).
- PostgreSQL 18 en ejecución y el archivo `.env` apuntando a la base de datos del proyecto.
- El repositorio `INF560-2026-02-Apellido` sin cambios pendientes.

> **CONVENCIÓN DE ESTA GUÍA**
> Todo bloque titulado **terminal** debe ejecutarse en la terminal, dentro de la carpeta del proyecto. Las líneas que empiezan con `$` son los comandos que usted escribe (sin el `$`); las líneas grises son la **salida esperada**, y las que empiezan con `#` son comentarios. No omita ningún bloque de terminal: cada uno es parte del procedimiento.

Paso 0.1 — Abrir el proyecto y revisar su estado

Abra una terminal (en VS Code: **Terminal → New Terminal**) y ubíquese en la carpeta de `eventos-app`. Compruebe que no tiene cambios sin confirmar y que todas las migraciones anteriores están aplicadas:

```
terminal                                                                Copiar

$ cd eventos-app
$ git status
On branch main
nothing to commit, working tree clean
$ git pull
Already up to date.
$ php artisan migrate:status
  Migration name ..................................... Batch / Status
  0001_01_01_000000_create_users_table ..................... [1] Ran
  ...
  xxxx_xx_xx_xxxxxx_add_user_id_to_eventos_table ............... [n] Ran
```

> **NOTA**
> Si `git status` muestra archivos modificados, confírmelos antes de empezar (`git add .` y `git commit -m "..."`). Si alguna migración aparece como **Pending**, ejecute `php artisan migrate` antes de continuar.

Paso 0.2 — Levantar los servidores de desarrollo

Necesita el servidor de PHP y el de Vite (para Tailwind) funcionando mientras trabaja. Elija **una** de las dos opciones:

```
terminal — opción A (un solo comando)                                   Copiar

$ composer run dev
# Inicia php artisan serve + Vite (y la cola) en la misma terminal.
# Deje esta terminal abierta y abra OTRA para los comandos de la guía.
```

```
terminal — opción B (dos terminales)                                    Copiar

# Terminal 1
$ php artisan serve
   INFO  Server running on [http://127.0.0.1:8000].
# Terminal 2
$ npm run dev
  VITE v7.x  ready in 500 ms
  ➜  Local:   http://localhost:5173/
```

Abra `http://127.0.0.1:8000/eventos` en el navegador y confirme que la aplicación carga. A partir de aquí, use una **tercera terminal** (o la segunda, en la opción A) para todos los comandos de la guía.

01 Teoría breve: Gates y Policies

Autenticar respondía «¿quién eres?». **Autorizar** responde «¿puedes hacer esto?». Laravel ofrece dos herramientas complementarias:

Herramienta	Qué es	Cuándo usarla
Gate	Un cierre (closure) con una regla suelta.	Permisos globales que no dependen de un modelo (p. ej. «entrar al panel»).

Herramienta	Qué es	Cuándo usarla
Policy	Una clase que agrupa las reglas de un modelo.	Autorizar acciones sobre un modelo (editar/borrar un evento).

DEFINICIÓN — POLICY

Una **Policy** es una clase con un método por acción (`update`, `delete`…). Cada método recibe el `User` y el modelo, y devuelve `true` o `false`. Laravel la **descubre automáticamente** por convención de nombres: `EventoPolicy` autoriza al modelo `Evento`.

El recorrido de una petición protegida queda así: middleware `auth` (¿hay sesión?) → **policy / gate** (¿tiene permiso?) → **controlador** (ejecuta la acción). Si cualquiera de los dos primeros filtros falla, el controlador nunca llega a tocar la base de datos.

ALTERNATIVA PARA RBAC COMPLETO

Para sistemas con muchos roles y permisos dinámicos, el paquete `spatie/laravel-permission` ofrece un RBAC completo con tablas de roles y permisos. Para `eventos-app` bastan las Policies y Gates integrados, que además enseñan el mecanismo base.

02 Ejercicio 1 — Un rol para los usuarios

Paso 1 — Crear la migración

Genere una migración que **modifique** la tabla `users` existente (no que cree una tabla nueva):

terminal Copiar

```
$ php artisan make:migration add_rol_to_users_table --table=users
   INFO  Migration [database/migrations/AAAA_MM_DD_HHMMSS_add_rol_to_users_table.php]
         created successfully.
```

La opción `--table=users` hace que el archivo generado ya traiga `Schema::table('users', ...)` en `up()` y en `down()`. El prefijo con la fecha y hora será distinto en su equipo.

Paso 2 — Escribir la migración

Abra el archivo recién creado en `database/migrations/` y reemplace su contenido completo por el siguiente:

database/migrations/AAAA_MM_DD_HHMMSS_add_rol_to_users_table.php Copiar

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            // 'usuario' | 'admin'
            $table->string('rol', 20)->default('usuario')->after('email');
        });
    }
```

```
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('rol');
        });
    }
};
```

> **NOTA**
> El `default('usuario')` hace que todos los usuarios que ya existen reciban el rol `usuario` al ejecutar la migración, y que los nuevos registros también lo tengan sin que el formulario de registro tenga que enviarlo. En PostgreSQL, `after()` se ignora (las columnas nuevas siempre van al final); se deja por compatibilidad con MySQL.

Paso 3 — Ejecutar la migración

terminal Copiar

```
$ php artisan migrate
   INFO  Running migrations.
  AAAA_MM_DD_HHMMSS_add_rol_to_users_table ..................... 15.21ms DONE
```

Compruebe desde la terminal que la columna existe, con el comando `db:table` de Artisan:

terminal Copiar

```
$ php artisan db:table users
  public.users ..............................................................
  Columns ..................................................... 9
  ...
  email ....................................................... varchar
  rol string, default: 'usuario'::character varying ...... varchar(20)
  ...
```

Paso 4 — Agregar el helper esAdmin() al modelo User

Abra `app/Models/User.php` y agregue el siguiente método **dentro** de la clase, debajo de los métodos que ya existen:

app/Models/User.php Copiar

```
    /**
     * Indica si el usuario tiene el rol de administrador.
     */
    public function esAdmin(): bool
    {
        return $this->rol === 'admin';
    }
```

> **SEGURIDAD — NO AGREGUE ROL A $FILLABLE**
>
> Si `rol` fuera asignable en masa, un usuario malicioso podría enviar un campo oculto `rol=admin` en el formulario de registro y convertirse en administrador (**escalada de privilegios por asignación masiva**). Por eso el rol **no** se agrega a `$fillable` (ni al atributo `#[Fillable]`, si su modelo lo usa): se asigna siempre de forma explícita, como en el seeder del paso siguiente.

Paso 5 — Crear un seeder con usuarios de prueba

Para probar la autorización necesitamos tres cuentas: un administrador y dos usuarios normales. En lugar de crearlas a mano, usaremos un **seeder**. Genérelo desde la terminal:

terminal Copiar

```
$ php artisan make:seeder UsuariosPruebaSeeder
   INFO  Seeder [database/seeders/UsuariosPruebaSeeder.php] created successfully.
```

Abra `database/seeders/UsuariosPruebaSeeder.php` y reemplace su contenido por:

database/seeders/UsuariosPruebaSeeder.php Copiar

```php
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;

class UsuariosPruebaSeeder extends Seeder
{
    public function run(): void
    {
        $usuarios = [
            ['name' => 'Administrador', 'email' => 'admin@uatf.edu.bo',
             'rol' => 'admin'],
            ['name' => 'Ana Dueña', 'email' => 'ana@uatf.edu.bo',
             'rol' => 'usuario'],
            ['name' => 'Luis Otro', 'email' => 'luis@uatf.edu.bo',
             'rol' => 'usuario'],
        ];

        foreach ($usuarios as $datos) {
            // Busca por email; si no existe, prepara uno nuevo (no duplica)
            $user = User::firstOrNew(['email' => $datos['email']]);

            $user->name = $datos['name'];
            $user->password = Hash::make('password');
            $user->email_verified_at = now();
            $user->rol = $datos['rol'];   // asignación explícita, no masiva
            $user->save();
        }
    }
}
```

Ejecute **solo** este seeder con la opción `--class`:

terminal Copiar

```
$ php artisan db:seed --class=UsuariosPruebaSeeder
   INFO  Seeding database.
```

> **NOTA**
>
> Gracias a `firstOrNew()`, puede ejecutar el seeder las veces que quiera sin duplicar usuarios. Si ya tenía una cuenta con alguno de esos correos, su contraseña pasará a ser `password` y su rol el indicado en el arreglo. La contraseña de las tres cuentas es `password`.

Paso 6 — (Opcional) Registrar el seeder en DatabaseSeeder

Para que `php artisan db:seed` (sin `--class`) también cree estas cuentas, agregue la llamada en `database/seeders/DatabaseSeeder.php`, dentro del método `run()`:

database/seeders/DatabaseSeeder.php Copiar

```
    public function run(): void
    {
        // ...seeders que ya tenga (categorías, etc.)
        $this->call(UsuariosPruebaSeeder::class);
    }
```

Paso 7 — Verificar los roles en PostgreSQL

Consulte la tabla directamente con `psql`. Reemplace `eventos_app` por el valor de `DB_DATABASE` y `postgres` por el de `DB_USERNAME` de su archivo `.env`; se le pedirá la contraseña de PostgreSQL:

terminal Copiar

```
$ psql -U postgres -d eventos_app -c "SELECT id, email, rol FROM users ORDER BY id;"
 id |         email          |   rol
----+------------------------+---------
  1 | estudiante@uatf.edu.bo | usuario
  2 | admin@uatf.edu.bo      | admin
  3 | ana@uatf.edu.bo        | usuario
  4 | luis@uatf.edu.bo       | usuario
(4 rows)
```

> **¿QUIERE QUE SU CUENTA SEA ADMINISTRADORA?**
>
> Hágalo con una sentencia SQL, también desde la terminal (cambie el correo por el suyo):
>
> `psql -U postgres -d eventos_app -c "UPDATE users SET rol = 'admin' WHERE email = 'su-correo@uatf.edu.bo';"`
>
> La respuesta esperada es `UPDATE 1`. Si obtiene `UPDATE 0`, el correo no coincide con ningún usuario.

Paso 8 — Confirmar el ejercicio en Git

terminal Copiar

```
$ git status
$ git add database/migrations app/Models/User.php database/seeders
$ git commit -m "feat(auth): columna rol, esAdmin() y seeder de prueba"
[main 1a2b3c4] feat(auth): columna rol, esAdmin() y seeder de prueba
 4 files changed, 62 insertions(+)
```

03 Ejercicio 2 — La EventoPolicy

Paso 1 — Generar la policy

terminal Copiar

```
$ php artisan make:policy EventoPolicy --model=Evento
   INFO  Policy [app/Policies/EventoPolicy.php] created successfully.
```

Si la carpeta `app/Policies` no existía, Artisan la crea. Con `--model=Evento`, el archivo trae métodos vacíos para cada acción (`viewAny`, `view`, `create`, `update`, `delete`, `restore`, `forceDelete`).

Paso 2 — Escribir las reglas

Abra `app/Policies/EventoPolicy.php` y reemplace **todo** su contenido por:

app/Policies/EventoPolicy.php Copiar

```php
<?php

namespace App\Policies;

use App\Models\Evento;
use App\Models\User;

class EventoPolicy
{
    // Se ejecuta ANTES que cualquier otro método: el admin puede todo
    public function before(User $user, string $ability): ?bool
    {
        return $user->esAdmin() ? true : null;
    }

    public function viewAny(User $user): bool
    {
        return true;   // todos los autenticados ven el listado
    }

    public function view(User $user, Evento $evento): bool
    {
        return true;   // todos pueden ver el detalle
    }

    public function create(User $user): bool
    {
        return true;   // cualquier usuario autenticado puede crear
    }

    public function update(User $user, Evento $evento): bool
    {
        return $user->id === (int) $evento->user_id;   // solo el dueño
    }

    public function delete(User $user, Evento $evento): bool
    {
        return $user->id === (int) $evento->user_id;   // solo el dueño
    }
}
```

EL TRUCO DE BEFORE(): TRUE, FALSE O NULL

Si `before()` devuelve `true` concede el permiso; si devuelve `false` lo **niega a todos** (¡incluido el dueño!). Por eso, para un usuario normal debe devolver `null`: «no decido aquí, sigan evaluando el método concreto». Así el admin pasa y el resto cae en la regla de propiedad.

NOTA

No hace falta registrar la policy: Laravel encuentra `EventoPolicy` para el modelo `Evento` por convención de nombres. El `(int)` en la comparación evita falsos negativos si el driver devuelve `user_id` como texto: con `===`, `5 === '5'` es `false`.

Paso 3 — Confirmar en Git

terminal Copiar

```
$ git add app/Policies/EventoPolicy.php
$ git commit -m "feat(auth): EventoPolicy con reglas por propiedad y bypass de admin"
```

04 Ejercicio 3 — Autorizar en el controlador

Paso 1 — Habilitar authorize() en el controlador base

Desde Laravel 11, el controlador base viene vacío y el helper `$this->authorize()` requiere el trait `AuthorizesRequests`. Abra `app/Http/Controllers/Controller.php` y déjelo así, para que todos los controladores lo hereden:

app/Http/Controllers/Controller.php Copiar

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Foundation\Auth\Access\AuthorizesRequests;

abstract class Controller
{
    use AuthorizesRequests;
}
```

Paso 2 — Verificar el permiso en cada acción

Abra `app/Http/Controllers/EventoController.php`. Agregue la línea de `authorize()` como **primera instrucción** de cada método indicado; el resto del código de cada método se mantiene igual:

app/Http/Controllers/EventoController.php Copiar

```php
public function create()
{
    $this->authorize('create', Evento::class);
    // ...código existente: mostrar el formulario
}

public function store(EventoRequest $request)
{
```

```
    $this->authorize('create', Evento::class);
    // ...código existente: crear con user_id = auth()->id()
}

public function edit(Evento $evento)
{
    $this->authorize('update', $evento);
    // ...código existente: mostrar el formulario de edición
}

public function update(EventoRequest $request, Evento $evento)
{
    $this->authorize('update', $evento);
    // ...código existente: guardar cambios
}

public function destroy(Evento $evento)
{
    $this->authorize('delete', $evento);
    // ...código existente: eliminar
}
```

En `create` y `store` se pasa la **clase** (`Evento::class`) porque el evento todavía no existe; en el resto se pasa la **instancia** `$evento`. Verifique que al inicio del archivo exista `use App\Models\Evento;`.

Si el usuario no cumple la regla, Laravel detiene la acción con un error `403 Forbidden` antes de tocar la base de datos. La policy es la **última línea de defensa**: aunque alguien envíe la petición a mano, no pasará.

Paso 3 — Comprobar que la ruta sigue registrada

terminal Copiar

```
$ php artisan route:list --path=eventos
  GET|HEAD   eventos ................. eventos.index › EventoController@index
  POST       eventos ................. eventos.store › EventoController@store
  GET|HEAD   eventos/create ........ eventos.create › EventoController@create
  GET|HEAD   eventos/{evento} ........ eventos.show › EventoController@show
  PUT|PATCH  eventos/{evento} ...... eventos.update › EventoController@update
  DELETE     eventos/{evento} ..... eventos.destroy › EventoController@destroy
  GET|HEAD   eventos/{evento}/edit .... eventos.edit › EventoController@edit
```

Si aparece algún error de sintaxis en lugar de la lista, corríjalo antes de seguir: `route:list` carga los controladores y lo detecta.

Paso 4 — Probar el 403 en el navegador

- **Prueba 1 —** Inicie sesión como `ana@uatf.edu.bo` (contraseña `password`) y cree un evento llamado «Evento de Ana». Cierre sesión.
- **Prueba 2 —** Inicie sesión como `luis@uatf.edu.bo` y cree «Evento de Luis».
- **Prueba 3 —** Averigüe el `id` del evento de Ana con la consulta de la terminal de abajo. Con la sesión de Luis, escriba en la barra de direcciones `http://127.0.0.1:8000/eventos/ID/edit` (ID = el de Ana). Debe ver la página **403 | This action is unauthorized**.
- **Prueba 4 —** Abra la misma URL con el `id` del evento de Luis: el formulario debe cargar con normalidad.

```
terminal Copiar
$ psql -U postgres -d eventos_app -c "SELECT id, user_id FROM eventos ORDER BY id;"
 id | user_id
----+---------
  7 |       3
  8 |       4
(2 rows)
```

Paso 5 — Confirmar en Git

```
terminal Copiar
$ git add app/Http/Controllers/Controller.php
$ git add app/Http/Controllers/EventoController.php
$ git commit -m "feat(auth): autorizar acciones de EventoController"
```

05 Ejercicio 4 — Ocultar acciones con @can

Proteger el controlador evita el acceso, pero es mejor experiencia **no mostrar** botones que el usuario no puede usar. La directiva `@can` consulta la misma policy desde la vista.

Paso 1 — Proteger los botones del listado

En `resources/views/eventos/index.blade.php`, localice los enlaces de **Editar** y el formulario de **Eliminar** de cada tarjeta y envuélvalos con `@can` como se muestra:

```
resources/views/eventos/index.blade.php Copiar
<div class="mt-3 flex items-center gap-3">
    @can('update', $evento)
        <a href="{{ route('eventos.edit', $evento) }}" class="text-sm text-blue-600">
            Editar
        </a>
    @endcan

    @can('delete', $evento)
        <form action="{{ route('eventos.destroy', $evento) }}" method="POST"
              onsubmit="return confirm('¿Eliminar este evento?')">
            @csrf
            @method('DELETE')
            <button class="text-sm text-red-600">Eliminar</button>
        </form>
    @endcan
</div>
```

Paso 2 — Proteger el botón «Nuevo evento» y la vista de detalle

Envuelva el enlace a `eventos.create` con `@can('create', App\Models\Evento::class)` … `@endcan`. Si su proyecto tiene `resources/views/eventos/show.blade.php`, aplique ahí los mismos `@can` del paso anterior a los botones Editar/Eliminar.

resources/views/eventos/index.blade.php Copiar

```
@can('create', App\Models\Evento::class)
    <a href="{{ route('eventos.create') }}"
       class="rounded bg-blue-600 px-4 py-2 text-white">
        Nuevo evento
    </a>
@endcan
```

Paso 3 — Limpiar vistas compiladas y probar

terminal Copiar

```
$ php artisan view:clear
   INFO  Compiled views cleared successfully.
```

Recargue `/eventos` con la sesión de Luis: debe ver **Editar** y **Eliminar** solo en «Evento de Luis». Con la sesión de Ana, solo en el suyo.

> **NOTA**
> Ahora cada usuario ve «Editar» y «Eliminar» solo en **sus** eventos, y el administrador los ve en todos. La vista y el controlador usan la **misma** regla: si cambia la policy, ambos se actualizan.

Paso 4 — Confirmar en Git

terminal Copiar

```
$ git add resources/views/eventos
$ git commit -m "feat(ui): ocultar acciones de eventos con @can según la policy"
```

06 Trabajo independiente

Haga que el catálogo de `categorias` sea **solo para administradores**: cualquiera crea eventos, pero solo un admin define las categorías. Además, enriquezca el panel de administración.

Consigna

- Proteja la gestión de `categorias` (crear/editar/eliminar) con el Gate `administrar`: middleware `can:administrar` en las rutas y `@can` en las vistas.
- Oculte los enlaces de gestión de categorías a los usuarios no administradores.
- En el panel `/admin`, agregue una tabla con **cada usuario**, su rol y la cantidad de eventos que creó (sugerencia: `User::withCount('eventos')`, que requiere la relación `hasMany` en `User`).
- Amplíe `UsuariosPruebaSeeder` con un cuarto usuario normal y ejecútelo de nuevo con `php artisan db:seed --class=UsuariosPruebaSeeder`. Toda creación de datos debe hacerse por seeder o por la interfaz, **no con Tinker**.

Entregables

- Commits atómicos (uno por cambio lógico) y `git push` a `INF560-2026-02-Apellido`.
- Capturas: la misma lista de eventos vista por el dueño, por otro usuario y por el administrador; el 403 al intentar entrar a `/admin` como usuario normal; y la salida de `php artisan route:list --path=categorias -v`.

07 Checklist de verificación

- ☐ La migración `add_rol_to_users_table` aparece como **Ran** en `php artisan migrate:status` y tiene `down()`.
- ☐ `php artisan db:table users` muestra la columna `rol` con valor por defecto `usuario`.
- ☐ `User::esAdmin()` existe y `rol` **no** está en `$fillable`.
- ☐ `UsuariosPruebaSeeder` crea las tres cuentas y la consulta `psql` muestra sus roles.
- ☐ `EventoPolicy` define `create`, `update` y `delete`, y `before()` devuelve `true` para el admin y `null` para el resto.
- ☐ El controlador base usa `AuthorizesRequests` y las cinco acciones llaman a `$this->authorize()`.
- ☐ Un usuario que no es dueño recibe 403 al editar o borrar un evento ajeno.
- ☐ El administrador puede editar y borrar cualquier evento.
- ☐ Las vistas ocultan Editar/Eliminar con `@can` según la policy.
- ☐ El Gate `administrar` protege `/admin` (visible en `route:list -v`) y el enlace se muestra solo a admins.
- ☐ Trabajo independiente: categorías solo para administradores y panel ampliado, en GitHub.

08 Problemas frecuentes

Síntoma	Causa y solución
El dueño también recibe 403	`before()` devuelve `false` para no-admin; debe devolver `null`. Revise también que la comparación use `(int) $evento->user_id`.
`Call to undefined method ...authorize()`	Falta `use AuthorizesRequests;` en `app/Http/Controllers/Controller.php`.
La policy nunca se aplica	El nombre no sigue la convención (debe ser `EventoPolicy` en `App\Policies`) o el modelo no está en `App\Models`.
`@can` siempre es false	La regla de la policy está invertida o el evento no tiene `user_id` (eventos creados antes de la GL09). Revíselo con la consulta `psql` de eventos.
Todos entran al panel admin	La ruta no tiene `->middleware('can:administrar')`. Verifique con `php artisan route:list --path=admin -v`.
El admin recibe 403 en /admin	El Gate no está en `boot()`, o el rol no quedó como `admin`: verifíquelo con la consulta `psql` del Ejercicio 1.
403 al crear un evento	El método `create()` de la policy no existe o no devuelve `true`.
`Target class [Database\Seeders\UsuariosPruebaSeeder] does not exist`	Nombre de clase o namespace distinto al del archivo. Corríjalo y ejecute `composer dump-autoload`.
`SQLSTATE[42701]: column "rol" ... already exists`	La columna ya fue creada. Revise `php artisan migrate:status`; si necesita rehacerla, use `php artisan migrate:rollback --step=1` y vuelva a migrar.
`psql` no se reconoce como comando	La carpeta `bin` de PostgreSQL 18 no está en el PATH. Agréguela, o ejecute la misma consulta en la **Query Tool** de pgAdmin.
Los cambios no se reflejan	Ejecute `php artisan optimize:clear` y recargue con Ctrl+F5.

10 Rúbrica de evaluación

Criterio	Pts
Migración del rol (up/down) ejecutada y helper `esAdmin()` en `User`	10
Seeder de usuarios de prueba ejecutado y verificado en PostgreSQL (sin Tinker)	10
`EventoPolicy` con `create`, `update` y `delete` correctos	15
`before()` correcto (admin `true`, resto `null`)	10
`authorize()` aplicado en create, store, edit, update y destroy	15
`@can` ocultando acciones en las vistas	10
Gate `administrar`, `AdminController` y panel protegido	15
Trabajo independiente: categorías solo para admins y panel ampliado	10
Commits atómicos y push a GitHub	5
Total	**100**